

21CSC201J — Data Structures & Algorithms

Unit 1 — All Important Theory

Definitions • Concepts • Derivations • Comparison Tables

Contents

1. Programming in C — Basics

2. Structures

3. Structures and Functions

4. Self-Referential Structures

5. Dynamic Memory Allocation (DMA)

6. Data Structure — Definition & Types

7. Time and Space Complexity

8. Asymptotic Notations

1. Programming in C — Basics

C was developed by **Dennis Ritchie** at **Bell Labs**, in the early **1970s**.

A C program consists of one or more **functions**, where a function is a group of statements that perform a well-defined task.

Primitive Data Types

A data type determines the set of values a data item can take and the operations that can be performed on it. C provides 4 basic data types:

Type	Size	Stores
char	1 byte	characters
int	2 bytes	integer numbers
float	4 bytes	floating point numbers
double	8 bytes	large floating point numbers

C also supports **4 modifiers**: two sign specifiers (`signed` , `unsigned`) and two size specifiers (`long` , `short`).

e.g. `unsigned int` , `unsigned short int`

2. Structures

Structure: A structure is similar to a record — it stores information about an entity, and can hold members of different data types.

Array vs Structure: An array stores elements of the **same** data type, whereas a structure stores information of **various** data types.

Declaration

```
struct structure_name
{
    data_type var_name;
    data_type var_name;
    ....
};
```

All variables of the structure are declared within the structure. Once declared, the structure becomes a **user-defined data type**. Each variable declared inside it is called a **member** of the structure.

Important: Memory is allocated for the structure only when a variable of the structure is declared — not at the time the structure itself is defined.

```
struct student stud1; // user-defined data type variable (memory now allocated)
```

A variable of the structure can also be declared at the time of structure declaration itself:

```
struct student
{
    ...
} stud1, stud2;
```

Typedef Declarations

The **typedef** keyword enables the programmer to create a new (alternate) name for an existing data type.

```
typedef existing_data_type new_data_type;
typedef int INTEGER;
INTEGER num; // same as: int num;
```

When **struct** is preceded by **typedef**, the struct itself becomes a new type name, so you no longer need to write **struct student** every time — just **student**.

Initialization of Structures

Initialization means assigning constants to the members of the structure at declaration.

- If the user does not initialize, C automatically does it: **int/float with 0**, and **string with '\0'**.
- Initializer values must match the corresponding member's data type and order in the structure definition.
- **Partial initialization:** if fewer values are given than members, the first few members are initialized and the remaining members are given default values.

```
struct student stud1 = {10, "Rahul"};
struct student stud3 = {30}; // name defaults to "\0"
```

Accessing Members of a Structure

A structure member variable is accessed using the **dot (.) operator**:

```
struct_var . member_name
```

Example: `stud1.r_no = 10;` `scanf("%d", &stud1.r_no);` `printf("%d", stud1.r_no);`

Arrays of Structures

An array of structures is used to declare multiple variables of the same structure type, and is declared the same way as an array of a built-in type:

```
struct struct_name
{
    ...
};
struct struct_name struct_var[index];
```

To assign values to the i-th element: `stud[i].r_no = 10;` `stud[i].fees = 10000;`

3. Structures and Functions

A function may access the members of a structure in **3 ways**:

- **Passing individual members** — each member is passed to the function as an ordinary variable; the called function has no idea it originally came from a structure.
- **Passing the entire structure** — like any other variable, the whole structure can be passed as a function argument (call by value — a *copy* is passed).
- **Passing the address of the structure** (structure and pointers) — the structure's address is passed so the function can modify the original data directly.

Why pass by pointer? Passing large structures to functions using the call-by-value method is very inefficient (it copies the entire structure). Therefore, it is preferred to pass structures through **pointers**.

Pointer to Structure — Syntax

```
struct struct_name
{
    data_type var1;
    ...
} *ptr;

struct student stud, *ptr_stud;
ptr_stud = &stud;    // assign address using & operator
```

Two equivalent ways to access members through a pointer:

Method	Syntax	Meaning
1	(*ptr_stud).r_no	brackets required — dot has higher precedence than *
2	ptr_stud -> r_no	arrow operator — preferred, cleaner ("pointing to")

4. Self-Referential Structures

Self-Referential Structure: A structure that contains a reference (pointer) to data of its own same type. It contains a pointer to data that is of the same type as the structure itself.

```
struct node
{
    int data;
    struct node *ptr;
};
```

The structure contains **2 kinds of data**: an integer `data`, and a pointer `ptr` which points to the memory location where another `struct node` is present.

This is the fundamental building block of **linked lists, trees, and graphs**. Chain of nodes:
`[data|ptr] → [data|ptr] → [data|ptr] → [data|NULL]`

5. Dynamic Memory Allocation (DMA)

Static Memory Allocation	Dynamic Memory Allocation
Number of memory blocks allocated is fixed	Memory can be increased/decreased during execution
Done at compile time	Done at run time
Memory can't be changed while the program runs	Memory changes as the program runs

Dynamic memory allocation is done using **4 functions** from the `stdlib.h` header file:

Function	Purpose
<code>malloc()</code>	allocates a single block of requested memory
<code>calloc()</code>	allocates multiple blocks of requested memory
<code>realloc()</code>	reallocates memory previously occupied by <code>malloc()</code> / <code>calloc()</code>
<code>free()</code>	frees (releases) dynamically allocated memory

(i) `malloc()` Function

Memory allocated by `malloc()` will have **garbage values** (not initialized).

```
ptr = (cast_type *) malloc(byte_size);

// e.g.
ptr = (int *) malloc(5 * sizeof(int)); // allocates 20 bytes
```

If space is insufficient, allocation **fails and returns a NULL pointer** — this should always be checked.

(ii) `calloc()` Function

Memory allocated by `calloc()` will be **initialized with zero** — this is the key difference from `malloc()`.

```
ptr = (cast_type *) calloc(number, byte_size);

// e.g.
ptr = (int *) calloc(5, sizeof(int)); // 5 blocks of 4 bytes = 20 bytes
```

(iii) `free()` Function

Used to dynamically de-allocate the memory allocated using `malloc()` and `calloc()`, since this memory will **not be de-allocated on its own** — it must be freed explicitly by the programmer.

```
free(ptr);
```

(iv) `realloc()` Function

Dynamically changes (resizes) the memory allocation of previously allocated memory. If memory previously allocated with `malloc` or `calloc` is insufficient, `realloc()` can be used to expand it.

```
ptr = realloc(ptr, newSize);

// e.g.
```



```
ptr = (int *) malloc(5 * sizeof(int));    // 20 bytes
ptr = realloc(ptr, 10 * sizeof(int));     // now 40 bytes
```

6. Data Structure — Definition & Types

Data Structure: A crucial part of data management. It contains a group of data elements and defines a particular way of storing and organizing data in a computer so it can be used efficiently.

Applications: Compiler design, Artificial Intelligence, Operating Systems, DBMS, Simulation, Numerical & Statistical analysis packages.

Classification of Data Structures

Primitive Data Structure — basic data types: int, float, char.

Non-Primitive Data Structure — divided further into:

Linear	Non-Linear
Elements stored in sequential (linear) order	Elements stored in non-sequential (hierarchical) order
Examples: Arrays, Linked List, Stacks, Queue	Examples: Tree, Graph

Linear Data Structures

- **Array** — a collection of data elements of the same data type; stored in **contiguous memory locations** and referenced by **index** (starts from 0 in C).
- **Linked List** — flexible, dynamic data structure. Elements are stored in **nodes**, arranged in sequential order. Each node points to the next node; the last node contains a **NULL** pointer to indicate the end of the list.
- **Stack** — a linear data structure in which insertion and deletion of elements are done only at **one end**, called the **top of stack**.
- **Queue** — a linear data structure in which an element is inserted at one end called the **rear**, and removed from the other end called the **front**.

Non-Linear Data Structures

- **Tree** — a non-linear data structure consisting of a collection of nodes arranged in a **hierarchical order**. One node is designated as the **root node**. The simplest form is a **binary tree**, where each node can have a maximum of 2 children. It follows a **parent-child relationship**.
- **Graph** — a non-linear data structure which is a collection of **vertices (nodes)** and **edges** that connect vertices. A graph is a **generalization of the tree structure**.

Operations on Data Structures

Operation	Purpose
Traversing	to access each data item exactly once
Searching	to find the location of a data item
Inserting	to add a new data item
Deleting	to remove a particular data item

Abstract Data Type (ADT)

ADT: Focuses on **what** it does and ignores **how** it does it. It can be a structure considered without regard to its implementation — a description of data in a structure with a list of operations that can be performed on the data within that structure.

The end user is **not concerned** with the details of how the methods carry out their task. They are only aware of the methods available to them — they call the methods and get the results, without concern for how they work internally.

Examples: Stack ADT, Queue ADT

7. Time and Space Complexity

Analyzing an algorithm means determining the amount of resources (such as time and memory) needed to execute it. The efficiency (complexity) of an algorithm is stated in terms of **time and space complexity**.

Time Complexity: The running time of a program as a function of the input size.

Space Complexity: The amount of computer memory required during program execution, as a function of the input size.

Best-Case, Average-Case, and Worst-Case

Case	Meaning
Worst case	Algorithm takes maximum time for execution — needs to perform the maximum number of steps/operations to process the input for accomplishing the given task. Gives an upper bound on the algorithm.
Best case	Algorithm takes minimum time for execution. Represents a lower bound on the algorithm.
Average case	Neither the best nor the worst case — indicates the average performance of the algorithm.

Example — Linear Search

```
for (i = 0; i < n; i++)
    if (a[i] == key)
        return i;
else
    return -1;
```

- **Worst case:** element is not present, or is the last element in the list $\rightarrow T(n) = n$
- **Best case:** element is in the first position $\rightarrow T(n) = 1$
- **Average case:** element could be anywhere in the list $\rightarrow T(n) \approx n$

Average case derivation:

```
P = 1/n
= 1.(1/n) + 2.(1/n) + 3.(1/n) + .... + n.(1/n)
= (1/n)(1 + 2 + 3 + .... + n)
= (1/n) . [n(n+1)/2]
= (n+1)/2 ≈ n/2
```

Time-Space Trade-Off

The best algorithm to solve a particular problem is one that requires less memory space and takes less time to complete its execution — but often both cannot be minimized together.

- If **space** is a big constraint, choose an algorithm that takes **less space** at the cost of more CPU time.
- If **time** is a major constraint, choose an algorithm that takes **minimum time** to execute at the cost of more space.

Time and space complexity can be expressed using a function $f(n)$, where n is the input size for a given instance of the problem being solved.

Algorithm Efficiency — Loop Analysis

The running time of an algorithm is based on the number of instructions it contains. If the algorithm contains loops, efficiency varies based on the number of loops and the running time of each loop.

Loop Type	Example	$f(n)$
Linear loop	<code>for(i=0;i</code>	n
Linear loop (step 2)	<code>for(i=0;i < 100;i+=2)</code>	$n/2$
Logarithmic loop	<code>for(i=1;i</code>	$\log n$
Nested — Linear $\times$ Logarithmic	outer loop n , inner loop $\log n$	$n \cdot \log n$
Nested — Quadratic	outer loop n , inner loop n (equal iterations)	n^2

8. Asymptotic Notations

Asymptotic Analysis: A kind of algorithm analysis with larger values of input data. Notations are used to represent the time complexities, given using functions.

Comparison of functions — growth rate order of time complexity of any algorithm:

$$1 < \log n < n < n \cdot \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$$

5 Notations: Big-Oh (O), Big-Omega (Ω), Big-Theta (Θ), little-oh (o), little-omega (ω)

1. Big-Oh Notation — O()

Specifies the **upper bound** of an algorithm.

Definition: The function $f(n) = O(g(n))$ **if and only if** there exists a positive constant **c** and **n_0** such that:

$$f(n) \leq c \cdot g(n), \quad \text{for all } n \geq n_0$$

Example 1: Let $f(n) = 2n+3$ and $g(n) = n$.

$$\begin{aligned} 2n+3 &\leq c \cdot n \quad \text{for } n \geq 1 \\ 2n+3 &\leq 5n \Rightarrow \text{true} \quad (\text{works for } c \geq 5) \\ 2n+3 &\leq 4n \Rightarrow \text{false} \\ 2n+3 &\leq 7n \Rightarrow \text{true} \\ \therefore f(n) &\leq c \cdot g(n) \text{ where } c \geq 5, g(n) = n \\ \therefore f(n) &= O(n) \end{aligned}$$

Example 2: Let $f(n) = 2n+3$ and $g(n) = n^2$.

$$\begin{aligned} 2n+3 &\leq 5n^2 \Rightarrow \text{true for } c \geq 5, n_0 \geq 1 \\ \therefore f(n) &= O(n^2) \end{aligned}$$

Note: if $f(n) = n$, $g(n)$ can be any valid upper bound (n , $n \log n$, n^2 , etc.) — Big-O is not unique.

2. Big-Omega Notation — Ω ()

Specifies the **lower bound** of an algorithm.

Definition: The function $f(n) = \Omega(g(n))$ **if and only if** there exists a positive constant **c** and **n_0** such that:

$$f(n) \geq c \cdot g(n), \quad \text{for all } n \geq n_0$$

Example: $f(n) = 2n+3$, $g(n) = n$

$$\begin{aligned} 2n+3 &\geq 1 \cdot n \quad \text{where } c \leq 5 \text{ \& } n \geq 1 \\ \therefore f(n) &= \Omega(n) \end{aligned}$$

3. Theta Notation — Θ ()

Specifies the **average case** analysis (tight bound).

Definition: The function $f(n) = \Theta(g(n))$ **if and only if** there exist positive constants **c_1** , **c_2** and **n_0** such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad (\text{lower}) \qquad (\text{upper})$$

Example: $f(n) = 2n+3$, $g(n) = n$

$$1 \cdot n \leq 2n+3 \leq 5 \cdot n \quad \text{where } c_1 = 1, c_2 = 5$$

$$\therefore f(n) = \theta(n)$$

4. Little-Oh Notation — $o()$

Used very rarely, for a small value of c .

Definition: The function $f(n) = o(g(n))$ if and only if there exists a positive constant value c and n_0 such that:

$$f(n) < c \cdot g(n)$$

5. Little-Omega Notation — $\omega()$

Definition: The function $f(n) = \omega(g(n))$ if and only if there exists a positive constant value c and n_0 such that:

$$f(n) > c \cdot g(n)$$

Quick Reference

Notation	Bound Type	Represents	Inequality
O	Upper bound	Worst case	$f(n) \leq c \cdot g(n)$
Ω	Lower bound	Best case	$f(n) \geq c \cdot g(n)$
Θ	Tight bound	Average case	$c_1 g(n) \leq f(n) \leq c_2 g(n)$
o	Strict upper bound	—	$f(n) < c \cdot g(n)$
ω	Strict lower bound	—	$f(n) > c \cdot g(n)$